Task-1: Highly Available Web Application on AWS

Objective

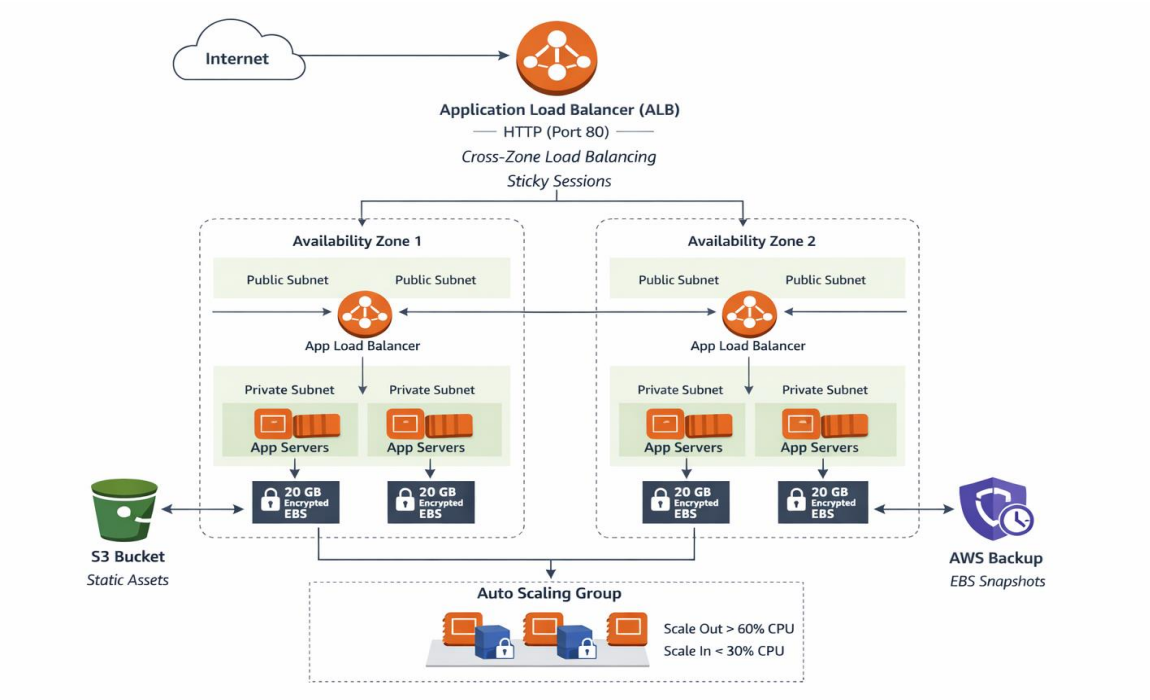
The objective of this project is to design and deploy a **highly available, scalable, and fault-tolerant web application** on AWS.

This includes:

- Distributing infrastructure across multiple Availability Zones
- Ensuring application availability using Load Balancer and Auto Scaling
- Securing and managing data using EBS and S3
- Implementing backup and recovery using AWS Backup

🗺 Architecture Diagram

Architecture Flow:



Internet → Application Load Balancer → EC2 Instances (Auto Scaling, Private Subnets) → EBS Storage → S3 (Static Content) → AWS Backup

1. Networking Infrastructure (VPC)

The foundation of this architecture is a custom VPC designed for high availability across multiple Availability Zones.

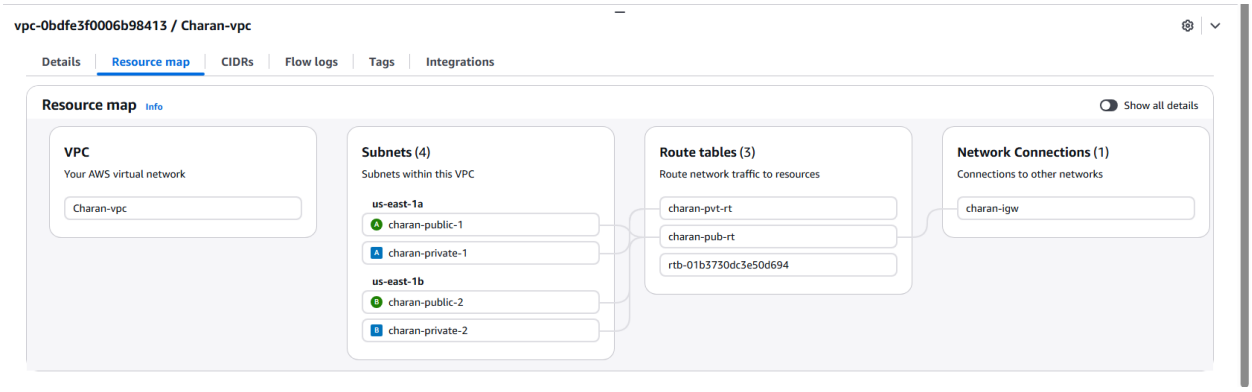
Configuration Details:

- Availability Zones: 2 AZs
- Subnets:
  - 2 Public Subnets (for Load Balancer)
  - 2 Private Subnets (for Application Instances)

Explanation:

This setup ensures fault tolerance by distributing resources across multiple AZs. Public subnets handle incoming traffic, while private subnets securely host backend application servers.

Visual Verification:



2. EC2 Deployment using Golden AMI

To ensure consistent deployments and faster scaling, a custom Golden AMI is used.

Pre-configuration:

- Apache/Nginx installed
- Application code deployed via User Data

User Data:

```
#!/bin/bash

# Update and install dependencies

apt-get update -y

apt-get install -y nginx git

# Clean default web root

rm -rf /var/www/html/*
```

Clone application repository

git clone https://github.com/schoolofdevops/html-sample-app /var/www/html/

Set permissions

chown -R www-data:www-data /var/www/html/

chmod -R 755 /var/www/html/

Restart service

systemctl restart nginx

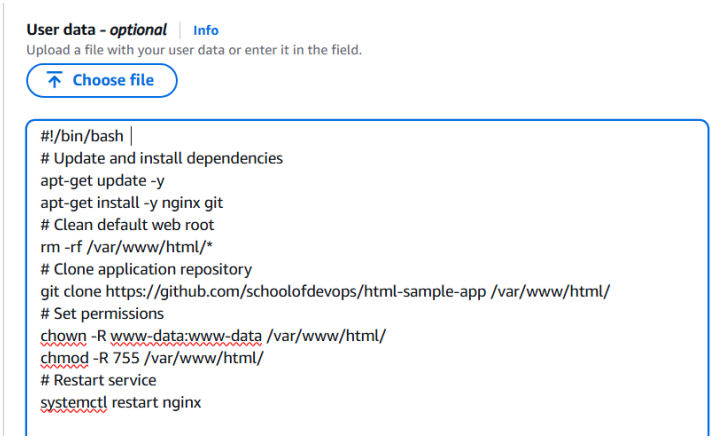
Deployment Details:

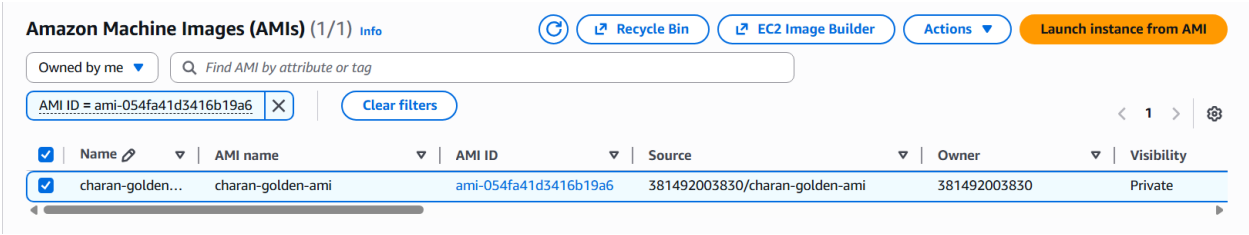
- Launch EC2 instances using Golden AMI
- Deploy instances in Public Subnets

Explanation:

Golden AMI helps standardize infrastructure by pre-installing required software and application code, reducing setup time during scaling events.

Visual Verification:





3. Storage Configuration (EBS)

Each application instance is configured with additional storage.

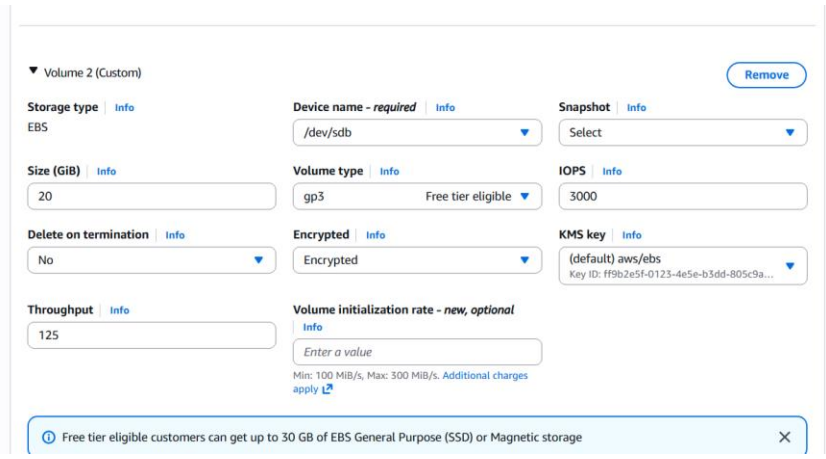
Configuration Details:

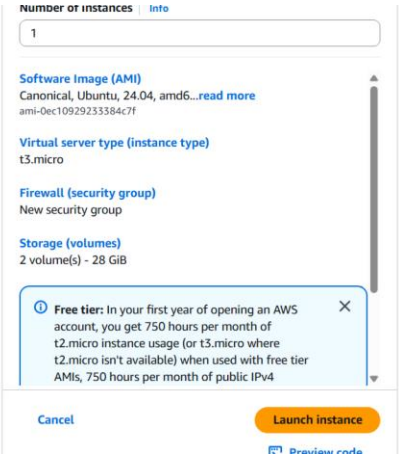
- Volume Type: gp3
- Size: 20 GB
- Encryption: Enabled using AWS KMS

Explanation:

EBS volumes provide persistent block storage. Encryption ensures that sensitive data is protected at rest.

Visual Verification:





4. Application Load Balancer (ALB)

The ALB distributes incoming traffic across multiple EC2 instances.

ALB Configuration:

- Protocol: HTTP
- Port: 80
- Deployment: Across 2 AZs

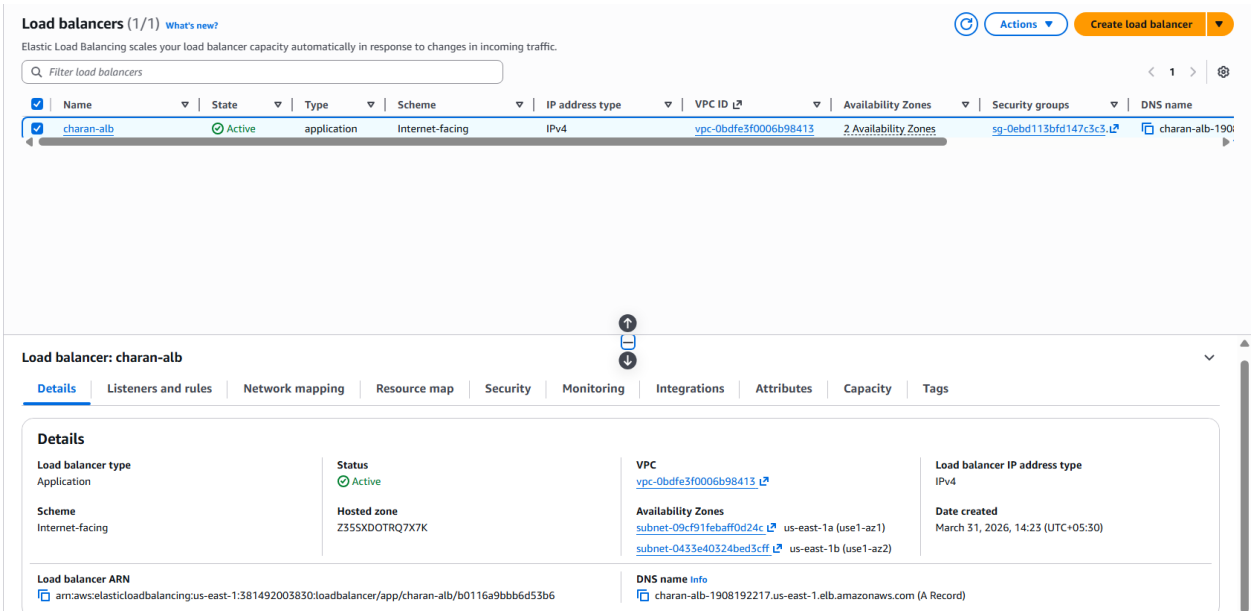
Attributes:

- Cross-Zone Load Balancing: Enabled
- Sticky Sessions: Enabled (1-hour duration)

Explanation:

The ALB ensures high availability and fault tolerance by routing traffic evenly. Sticky sessions help maintain session consistency for users.

Visual Verification:



5. Launch Template Setup

The Launch Template defines the configuration for EC2 instances used by Auto Scaling.

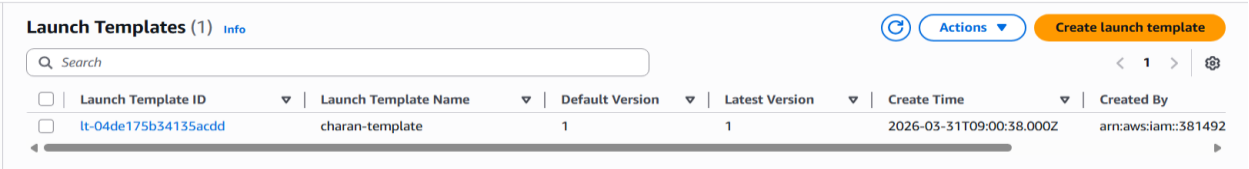
Configuration Details:

- AMI: Golden AMI
- Storage: 20 GB EBS (Encrypted)

Explanation:

Launch Templates simplify instance creation and ensure consistency during scaling operations.

Visual Verification:



6. Auto Scaling Group (ASG)

The Auto Scaling Group ensures application availability and handles scaling based on demand.

Scaling Parameters:

- Minimum Capacity: 2
- Maximum Capacity: 6
- Desired Capacity: 2

Scaling Policies:

- Scale Out: CPU > 60%
- Scale In: CPU < 30%
- Cooldown: 300 seconds

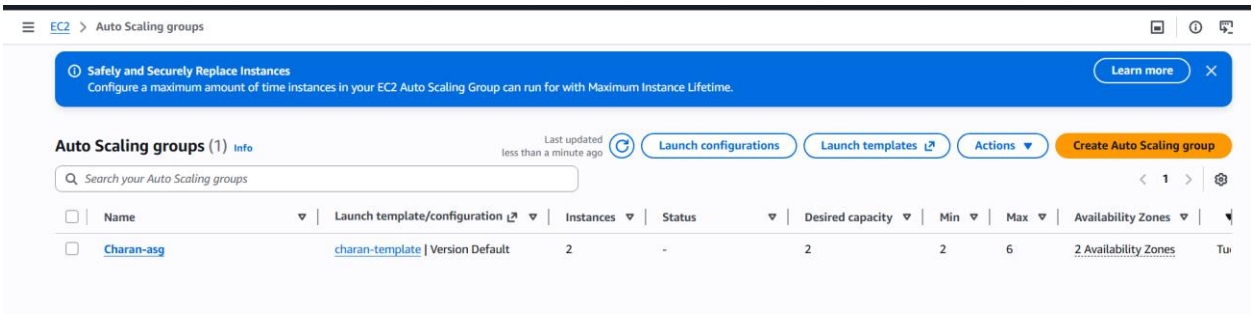
Termination Policy:

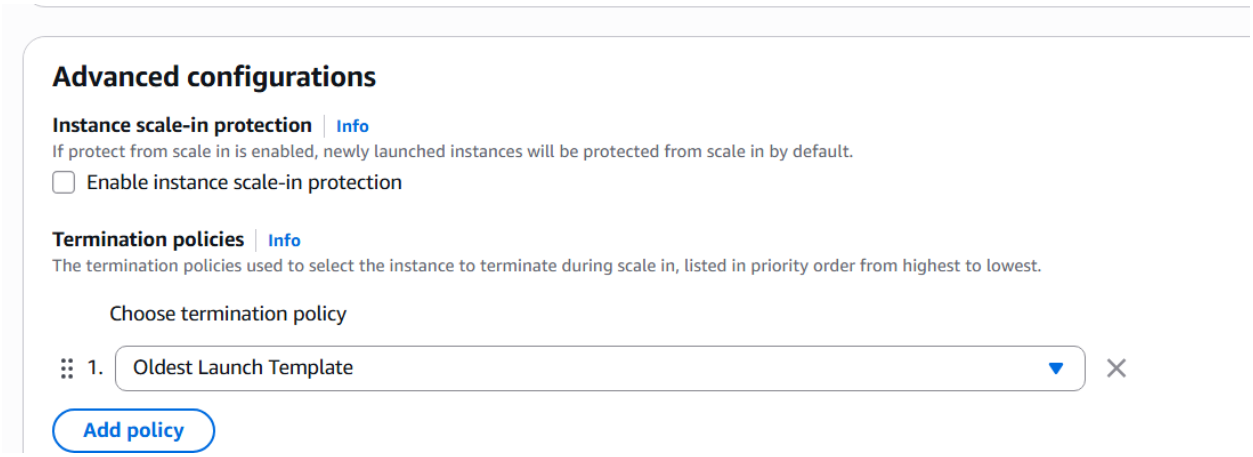
- OldestLaunchTemplate

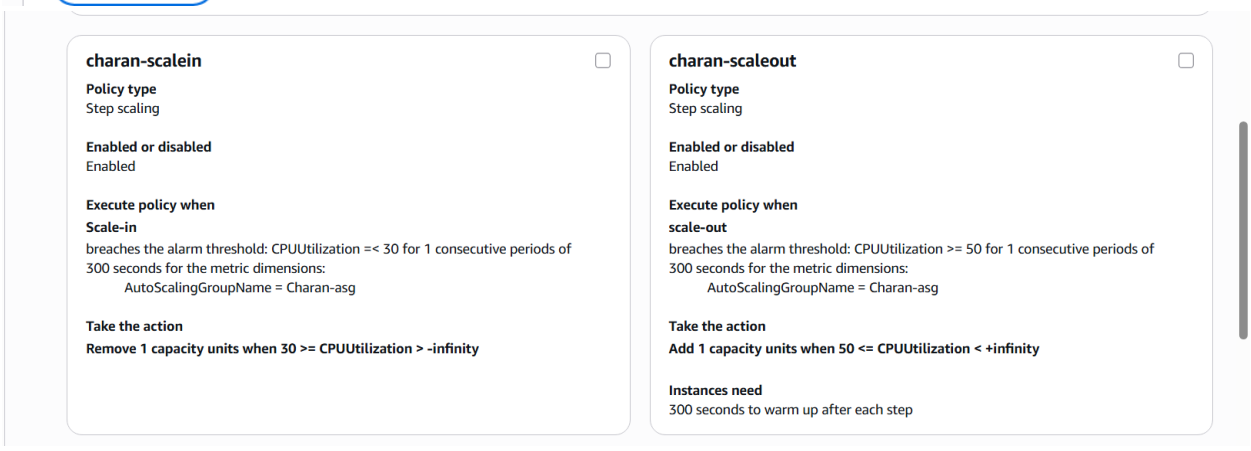
Explanation:

ASG automatically adjusts the number of instances based on load, ensuring performance and cost efficiency.

Visual Verification:







7. Storage (S3 & Backup)

S3 Configuration:

- Used for static assets
- Versioning: Enabled
- Encryption: SSE-S3

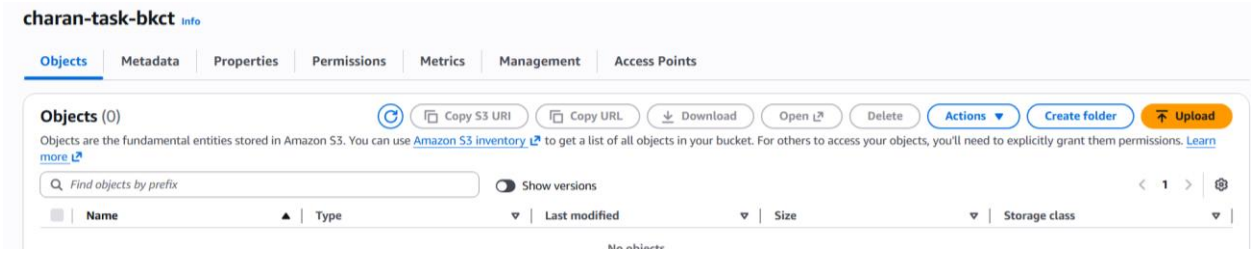
Backup Configuration:

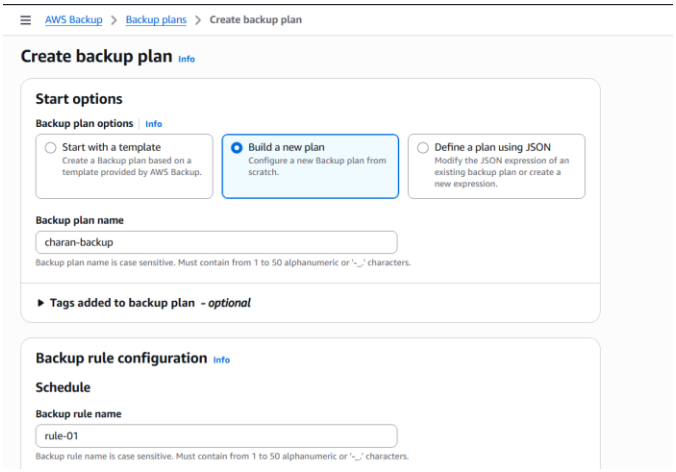
- EBS Snapshots: Daily
- Retention: 7 Days (AWS Backup Vault)

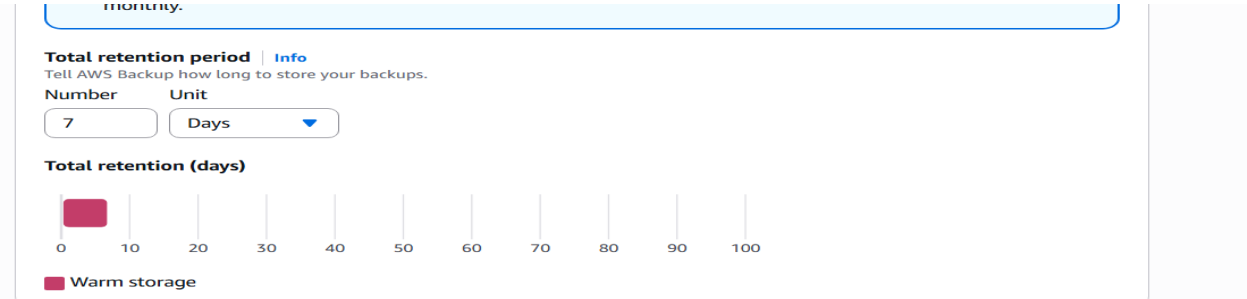
Explanation:

S3 provides scalable object storage, while AWS Backup ensures data protection and recovery.

Visual Verification:







8. Validation

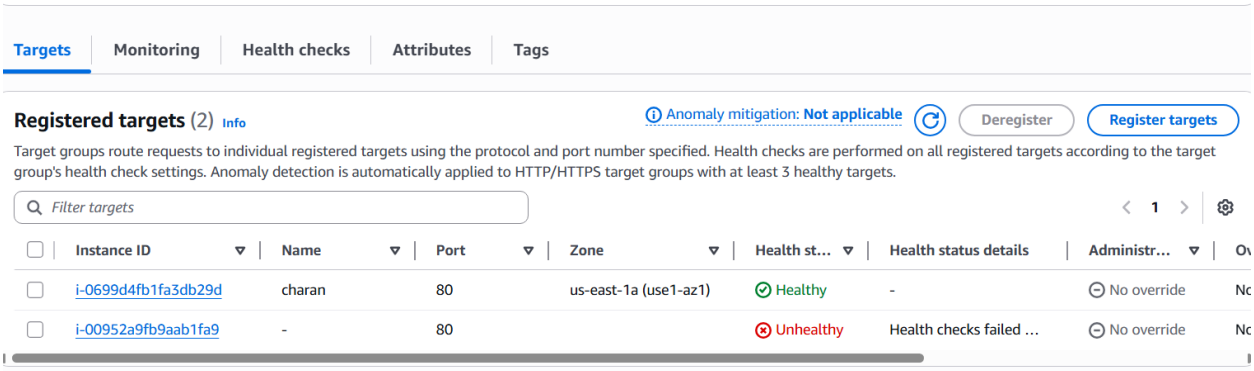
Checks Performed:

- ALB health checks pass for all instances
- Auto Scaling launches new instances under load
- Static assets load correctly from S3
- EBS snapshots are created successfully

Explanation:

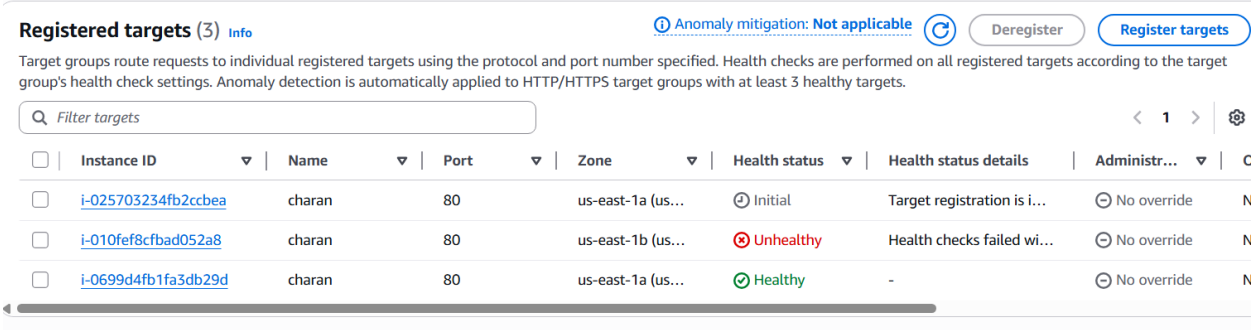
These validations confirm that all components are functioning correctly, and the system is highly available.

Visual Verification:



New target instance is registered, and health checks are started again for the new

instance.



Final Outcome

This architecture successfully implements:

- High Availability across multiple AZs
- Load-balanced traffic distribution
- Auto Scaling based on demand
- Secure and durable storage
- Automated backup and recovery